

Final Design Report

by

Just a Moment

Camden Lussier, Patrick LaLonde, Lucas Garcia Valdivia

EK301 Fall '25

Professor Katherine Zhang

December 5th, 2025

Introduction

Our final truss design was designed to achieve the best possible balance between price, simplicity, and durability. We focused on a structure that would handle large loads while maintaining a straightforward truss design that followed the design criteria, which can minimize unnecessary material use and building difficulty. We focused on incorporating the raised loading joint, increasing the lengths of our members to properly meet the length requirements, and calculated the smallest possible member lengths that would still meet the requirements, in order to make our design as strong as possible. Among these, our main design objective was member length. By increasing the lengths of our members, we were able to meet the requirements and design a truss that is durable. When adjusting the lengths, we recognized that buckling was the main way that members in compression fail, and the best approach to boost their load-bearing capability was to keep them as short as the requirements would allow. We avoided unnecessary height for the truss while recognizing that a shorter design would reduce vertical forces in members that are crucial for a load-bearing structure. Our attempt to optimize performance while maintaining the truss's affordability and durability is reflected in the final designs.

Procedure

Our design procedure underwent significant revision from the preliminary design report due to critical specification errors. Upon reviewing our preliminary designs, we discovered that neither of our initial truss configurations met the required specifications: specifically, our trusses failed to meet the required 32-inch span from support to support, and we had not raised the loading joint 2 inches above the horizontal baseline as required. These oversights necessitated a complete redesign process.

To address these issues, we first corrected the overall span to meet the 32-inch support-to-support requirement by recalculating joint positions and extending the horizontal dimensions of our truss geometry. We then raised the loading joint by 2 inches above the horizontal baseline to meet the project specifications. In doing so, we shortened two of our members below the 8.5 inch minimum length. In order to correct this, we increased the length of every member, so that our smallest members were 8.825 inches. With the geometry corrected to meet all requirements, we systematically adjusted member lengths throughout the structure to optimize the design.

Members 1, 2, 3, 4, 5, 8, and 11 were set to 10.5 inches, members 6 and 10 to 10.689 inches, and members 7 and 9 to 8.825 inches, ensuring compliance with all constraints while maintaining structural efficiency. After each geometric modification, we analyzed the design using our MATLAB program to calculate the critical buckling load (P_{crit}), identify the critical member, determine the total cost, and verify that the predicted failure load exceeded 32 oz under all uncertainty scenarios (nominal, strong, and weak cases). This revised procedure emphasized the importance of thoroughly verifying all design constraints before conducting detailed analysis, a lesson that proved valuable in developing a compliant and optimized final design.

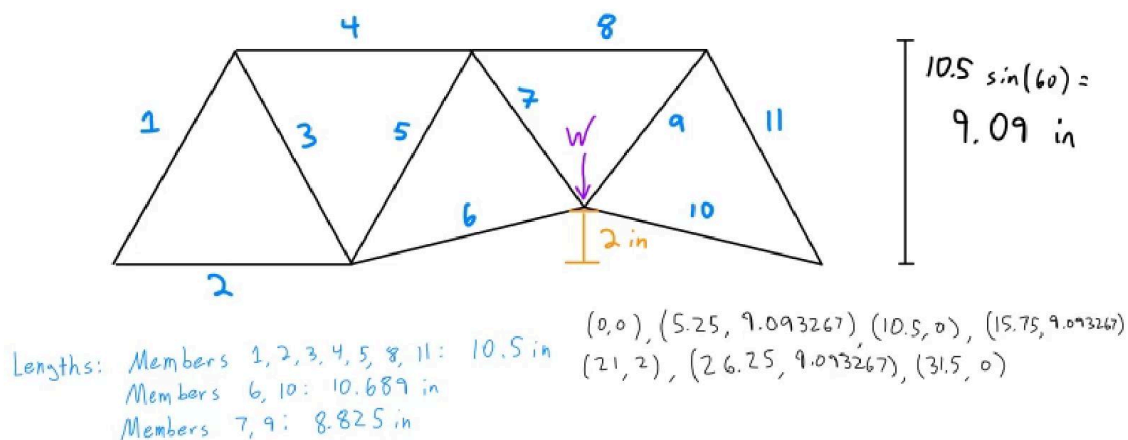
Analysis

Our analysis approach remained consistent with our preliminary design, using the same core formulas for buckling analysis and force calculations in MATLAB. However, we made significant design changes after discovering that both preliminary designs failed to meet the required 32-inch span and 2-inch loading joint elevation. This oversight called for a complete geometric redesign and taught us to thoroughly verify all constraints before detailed analysis.

The preliminary work also revealed that achieving the minimum 8.5-inch member length for interior diagonals was more challenging than anticipated, leading us to strategically utilize the required 2-inch joint elevation to increase members 7 and 9 to 8.825 inches. To account for the safety factor, we used the formula $\text{max_load}/\text{min_load} = 2.09$.

To account for uncertainty, we applied the formula $P_{\text{crit}} = 0.18 \times P_{\text{crit, nom}}$ to calculate the 18% variation in buckling force for each member, accounting for material variability and geometric imperfections. We analyzed three scenarios: nominal ($P_{\text{crit, nom}}$), strong ($P_{\text{crit, nom}} + P_{\text{crit}}$), and weak ($P_{\text{crit, nom}} - P_{\text{crit}}$) cases, propagating these uncertainties through our structural analysis to determine corresponding failure loads. This confirmed that even in the weak case, our design maintains adequate capacity well above the required 32 oz load.

Results



ExtrussTrussValues							
Member Number	Member Length (in)	Force Value (oz)	T or C	Pcrit (oz)	Pcrit Uncertainty (+/-)	Max Buckling Load (oz)	Max Buckling Load Uncertainty (+/-)
1	10.50	1.59441454	T	0.00	0.00	0.00	0.00
2	10.50	0.56390723	T	0.00	0.00	0.00	0.00
3	10.50	-1.51775032	C	38.42	6.98	38.42	6.98
4	10.50	1.27068022	T	0.00	0.00	0.00	0.00
5	10.50	1.80127914	T	0.00	0.00	0.00	0.00
6	10.69	-1.27720308	C	37.15	6.75	37.15	6.75
7	8.82	-2.03968128	C	51.76	9.40	51.76	9.40
8	10.50	3.09935445	T	0.00	0.00	0.00	0.00
9	8.82	-2.78012635	C	51.76	9.40	51.76	9.40
10	10.69	-1.01897171	C	37.15	6.75	37.15	6.75
11	10.50	2.32002720	T	0.00	0.00	0.00	0.00

Summary results:

Maximum load: 66.97 +/- 12.17 oz

Truss cost: \$182.53

Load-to-cost ratio: 0.3669 oz/dollar

Discussion & Conclusion

The final truss design's structural performance was evaluated based on the placement of a 32 oz concentrated load at joint 5, the raised loading point. The analysis results identify member 9 as the critical member, which is predicted to fail at a maximum applied load of 66.97 oz. This corresponds to a critical buckling load of 51.76 oz in member 9 itself. Our design achieved a total cost of \$182.53 with a load-to-cost ratio of 0.367 oz per dollar, reflecting our emphasis on balancing structural performance with economic efficiency. Our design process involved carefully balancing competing objectives by minimizing member lengths to improve buckling

resistance and reduce costs. After correcting our preliminary design to meet the required 34-inch span and 2-inch joint elevation, we strategically set most members to 10.5 inches, with members 6 and 10 at 10.689 inches due to geometric requirements, and members 7 and 9 at 8.825 inches just above the minimum. We intentionally kept the interior diagonals as short as possible to maximize their buckling resistance, yet member 9 still became critical due to the magnitude of compressive force it experiences, demonstrating that load distribution is equally important as member length in determining structural capacity.

The truss achieved a safety factor of 2.09 and a load-to-cost ratio of 0.367 oz per dollar, reflecting an economical design that prioritizes structural integrity without excessive material use. The most insightful element of this project was understanding that optimizing a truss requires analyzing which members carry the most critical loads rather than uniformly minimizing all member lengths. The iterative testing process reinforced that compression members, even when kept short to resist buckling, can still fail if force distribution concentrates too much load in them, emphasizing the importance of system-level thinking over isolated member optimization.

If we were to revise this project, we would explore whether increasing the loading joint height beyond the required 2 inches could reduce compression in member 9 by decreasing the angle of the interior diagonals. We would also investigate asymmetric configurations to strategically place more material in high-stress regions while reducing costs elsewhere, rather than maintaining our intuitive but potentially suboptimal symmetric design. Additionally, physical testing would validate our computational predictions and reveal real-world factors such as joint behavior and material imperfections that our idealized model does not capture. A more comprehensive parametric study could identify whether alternative geometries with different

member arrangements could achieve better performance at similar or lower cost. Our final design successfully shows how it is possible to create a structurally sound truss that meets all of the project requirements.

Seeing that we primarily focused on consolidating our member lengths while still meeting the requirements, we were able to meet all of the project constraints while achieving a safety factor of 2.09. This project shows that effective designs of structures, especially trusses, need in-depth analysis, critical thinking skills, and the ability to calculate trade-offs in real time. These critical skills learned can easily be applied to real-world applications, which are invaluable to structural engineers.

Appendix

Hartford Roof Collapse Discussion

Meeting Chair: Patrick L.

Meeting Recorder: Camden L.

Meeting Details: 12/1/25 at 12:20pm, Duration: 21 minutes.

Meeting Participants: Camden L., Lucas G., Patrick L.

Meeting Agenda

- Review material options and how Engineering Principle #1 should guide design decisions
- Establish an appropriate safety factor (the structure must safely exceed a 32 oz load)

Important Points

- Engineering Principle #1, (prioritizing health, safety, and public welfare), directly applies to our project.
- In connection with Principle #1, our group agreed that protecting users must outweigh cost, build speed, or efficiency. One way to follow this is to design as if the weakest scenario is the most likely one. (Camden)
- Material selection should focus on stiffness and stability, especially choosing pieces that have a low risk of buckling, helping to ensure trustworthy behavior. (Camden)
- A main issue in the Hartford case was too much dependence on the software's output without confirming results through hand calculations.
- We know that digital resources are essential, especially for complex structures like the Hartford Arena roof, but it's crucial to cross-check designs using multiple programs or another kind of failsafe. (Lucas)
- Engineers who approved the Hartford design using a flawed program are to blame
- They knew the risks, but people such as the construction crews, did not. (Lucas)
- For our team's project, a high safety factor means that we treat uncertainties seriously and we have all members review the final design, so that we are far less likely to make a mistake. (Lucas)
- A major part of Principle #1 is the idea that engineers must verify all results rather than depending on computers to do all of the work for you, ensuring that hand calculations can confirm the software's predictions. (Patrick)
- The principle also emphasizes reducing uncertainty, which means choosing designs that do not sit right at the limit of what they can hold. (Camden)

- As applied to our truss, we should be confident that it exceeds the required 32 oz minimum load with room to spare. (Lucas)
- Industry safety factors often start around 2.0, so if our truss needs to hold 32 oz, a perfect target failure load would be 64 oz or more.
- Seeing that this number may be difficult to reach in our project, aiming for as high a margin of safety as possible is good practice. (Patrick)

Final Truss Code

```
function runMyTruss()

    X = [0, 5.25, 10.5, 15.75, 21, 26.25, 31.5];
    Y = [0, 9.093267, 0, 9.093267, 2, 9.093267, 0];
    jointNames = {'1','2','3','4','5','6','7'};
    fprintf('=== Joint Coordinates ===\n');
    for i = 1:length(X)
        fprintf('  Joint %d (%s): (%.3f, %.3f)\n', ...
            i, jointNames{i}, X(i), Y(i));
    end
    fprintf('\n');
    % Members:
    % 1:1-2, 2:1-3, 3:2-3, 4:2-4, 5:3-4, 6:3-5,
    % 7:4-5, 8:4-6, 9:5-6, 10:5-7, 11:6-7
    C = [
        1 1 0 0 0 0 0 0 0 0 0; % Joint 1
        1 0 1 1 0 0 0 0 0 0 0; % Joint 2
        0 1 1 0 1 1 0 0 0 0 0; % Joint 3
        0 0 0 1 1 0 1 1 0 0 0; % Joint 4
        0 0 0 0 0 1 1 0 1 1 0; % Joint 5
        0 0 0 0 0 0 0 1 1 0 1; % Joint 6
        0 0 0 0 0 0 0 0 0 1 1 % Joint 7
    ];

    memberList = { ...
        '1-2','1-3','2-3','2-4','3-4','3-5', ...
        '4-5','4-6','5-6','5-7','6-7'};

    fprintf('=== Member Connections ===\n');
```

```

for m = 1:size(C,2)

    joints = find(C(:,m));

    fprintf('  Member %2d (%s): Joint %s to Joint %s\n', ...
        m, memberList{m}, jointNames{joints(1)}, jointNames{joints(2)});

end

fprintf('\n');

Sx = zeros(7,1);
Sy = zeros(7,1);

Sx(1) = 1;
Sy(1) = 1;
Sy(7) = 1;

fprintf('=== Supports ===\n');

fprintf('  Joint 1: pin (x,y)\n');
fprintf('  Joint 7: roller (y)\n\n');

L = zeros(14,1);
L(2*5) = -1;

fprintf('=== Load ===\n');

fprintf('  1 N downward at Joint 5 (21 in, 2 in)\n\n');

j = size(C,1);
m = size(C,2);

r = nnz(Sx) + nnz(Sy);

fprintf('=== Stability Check ===\n');

fprintf('  Joints j = %d\n', j);
fprintf('  Members m = %d\n', m);
fprintf('  Reactions r = %d\n', r);
fprintf('  2j = %d, m+r = %d → ', 2*j, m+r);

if 2*j == m + r

    fprintf('✓ Statically Determinate\n\n');

else

```

```
        fprintf('⚠ NOT Determinate\n\n');  
  
end  
  
groupName = 'Just a Moment';  
  
members    = 'Lucas Garcia, Camden Lussier, Patrick LaLodne';  
  
dateStr     = datestr('12/04/2025');  
  
fprintf('=== Running Solver ===\n\n');  
  
solveTruss(C, X, Y, Sx, Sy, L, groupName, members, dateStr);  
  
end
```

```

function solveTruss(C, X, Y, Sx, Sy, L, groupName, members, dateStr)

    [numJoints, numMembers] = size(C);

    numSupports = size(Sx, 2);

    jointLabels = {'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'};

    memberNames = cell(numMembers, 1);

    letterMap = cell(numMembers, 1);

    for m = 1:numMembers

        joints = find(C(:, m));

        if numel(joints) == 2

            memberNames{m} = [jointLabels{joints(1)} jointLabels{joints(2)}];

            letterMap{m} = memberNames{m};

        else

            memberNames{m} = sprintf('M%d', m);

            letterMap{m} = sprintf('%d', m);

        end

    end

    A = zeros(2*numJoints, numMembers + numSupports);

    row = 1;

    for j = 1:numJoints

        for m = 1:numMembers

            joints = find(C(:, m));

            if numel(joints) == 2

                a = joints(1);

                b = joints(2);

                if j == a

                    dx = X(b) - X(a);

                    dy = Y(b) - Y(a);

                    L_member = sqrt(dx*dx + dy*dy);

```

```

        A(row, m) = dx / L_member;

elseif j == b

    dx = X(a) - X(b);

    dy = Y(a) - Y(b);

    L_member = sqrt(dx*dx + dy*dy);

    A(row, m) = dx / L_member;

end

end

end

A(row, numMembers+1:numMembers+numSupports) = Sx(j, :);

row = row + 1;

for m = 1:numMembers

    joints = find(C(:, m));

    if numel(joints) == 2

        a = joints(1);

        b = joints(2);

        if j == a

            dx = X(b) - X(a);

            dy = Y(b) - Y(a);

            L_member = sqrt(dx*dx + dy*dy);

            A(row, m) = dy / L_member;

        elseif j == b

            dx = X(a) - X(b);

            dy = Y(a) - Y(b);

            L_member = sqrt(dx*dx + dy*dy);

            A(row, m) = dy / L_member;

        end

    end

end

```

```

        end

    end

    A(row, numMembers+1:numMembers+numSupports) = Sy(j, :);

    row = row + 1;

end

T = A \ L;

memberForces = T(1:numMembers);

reactionForces = abs(T(numMembers+1:end));

memberLengths = zeros(numMembers, 1);

for m = 1:numMembers

    joints = find(C(:, m));

    if length(joints) == 2

        j1 = joints(1);

        j2 = joints(2);

        dx = X(j2) - X(j1);

        dy = Y(j2) - Y(j1);

        memberLengths(m) = sqrt(dx^2 + dy^2);

    end

end

P_crit = 249 * exp(-0.178 * memberLengths); % in oz

R_squared = 0.967;

relative_uncert_percent = sqrt(1 - R_squared) * 100; % ~18.1%

P_crit_uncert = P_crit * (relative_uncert_percent / 100);

Rm = memberForces;

W_failure = zeros(numMembers, 1);

W_failure_uncert = zeros(numMembers, 1);

```

```

for m = 1:numMembers
    if memberForces(m) < 0
        P_crit_N = P_crit(m) / 3.59694;
        P_crit_uncert_N = P_crit_uncert(m) / 3.59694;

        W_failure(m) = P_crit_N / abs(Rm(m));

        W_failure_uncert(m) = W_failure(m) * (P_crit_uncert(m) / P_crit(m));
    else

        W_failure(m) = Inf;
        W_failure_uncert(m) = 0;
    end
end

[W_failure_c, criticalIdx] = min(W_failure);
W_failure_c_uncert = W_failure_uncert(criticalIdx);

W_failure_c_oz = W_failure_c * 3.59694;
W_failure_c_uncert_oz = W_failure_c_uncert * 3.59694;

fprintf('\n>> criticalMember\n');
fprintf('EK301, Group: Just a Moment, Members: Lucas Garcia, Camden Lussier,
Patrick Lalonde, %s\n', dateStr);

fprintf('Load: %.0f N\n', abs(sum(L)));

fprintf('\nMember Forces in N:\n');

```

```

for i = 1:numMembers

    f = memberForces(i);

    letter = letterMap{i};

    if abs(f) < 1e-6

        fprintf('m%s: %.2f N (Zero Force Member)\n', letter, f);

    else

        if f > 0

            type = 'T';

        else

            type = 'C';

        end

        fprintf('m%s: %.2f N (%s)\n', letter, f, type);

    end

end

fprintf('\nReaction Forces in N:\n');

reactionIdx = 1;

for i = 1:numSupports

    xJoint = find(Sx(:, i), 1);

    yJoint = find(Sy(:, i), 1);

    if ~isempty(xJoint) && ~isempty(yJoint) && xJoint == yJoint

        fprintf('Sx%d (Joint %s): %.2f N\n', i, jointLabels{xJoint},
reactionForces(reactionIdx));

        reactionIdx = reactionIdx + 1;

    elseif ~isempty(xJoint)

        fprintf('Sx%d (Joint %s): %.2f N\n', i, jointLabels{xJoint},
reactionForces(reactionIdx));

        reactionIdx = reactionIdx + 1;

    elseif ~isempty(yJoint)

```

```

        fprintf('Sy%d (Joint %s): %.2f N\n', i, jointLabels{yJoint},
reactionForces(reactionIdx));

        reactionIdx = reactionIdx + 1;

    end

end

fprintf('\n=== Critical Load Analysis ===\n');

fprintf('Critical Member: #d (%s)\n', criticalIdx, letterMap{criticalIdx});

fprintf('Member Length: %.2f in\n', memberLengths(criticalIdx));

fprintf('P_crit (buckling load): %.2f ± %.2f oz\n', P_crit(criticalIdx),
P_crit_uncert(criticalIdx));

fprintf('Force ratio Rm: %.4f N/N\n', Rm(criticalIdx));

fprintf('Critical Failure Load W_failure,c: %.2f ± %.2f N\n', W_failure_c,
W_failure_c_uncert);

fprintf('Critical Failure Load W_failure,c: %.2f ± %.2f oz\n',
W_failure_c_oz, W_failure_c_uncert_oz);


N_TO_OZ = 3.59694;

totalLength = sum(memberLengths);

totalLengthInches = totalLength;

trussStructureCost = totalLengthInches + 10 * numJoints;

totalLoadOz = abs(sum(L)) * N_TO_OZ;


failureLoadOz = W_failure_c_oz;

failureLoadOz_uncert = W_failure_c_uncert_oz;

loadToCostRatio = failureLoadOz / trussStructureCost;

loadToCostRatio_uncert = loadToCostRatio * (failureLoadOz_uncert /
failureLoadOz);

fid1 = fopen('ExtrussTrussValues.csv', 'w');

```

```

fprintf(fid1, 'ExtrussTrussValues\n');

fprintf(fid1, 'Member Number,Member Length (in),Force Value (oz),T or
C,Pcrit (oz),Pcrit Uncertainty (+-),Max Buckling Load (oz),Max Buckling Load
Uncertainty (+-)\n');

for i = 1:numMembers

    memberNum = i;

    lengthInch = memberLengths(i);

    forceOz = memberForces(i) * N_TO_OZ;

    if abs(memberForces(i)) < 1e-6

        tOrC = 'N/A';

        pcrit = 0;

        pcritUncert = 0;

        maxBuckling = 0;

        maxBucklingUncert = 0;

    elseif memberForces(i) > 0

        tOrC = 'T';

        pcrit = 0; % Tension members don't have P_crit for buckling

        pcritUncert = 0;

        maxBuckling = 0;

        maxBucklingUncert = 0;

    else

        tOrC = 'C';

        pcrit = P_crit(i);

        pcritUncert = P_crit_uncert(i);

        maxBuckling = pcrit;

        maxBucklingUncert = pcritUncert;

    end

```

```

        fprintf(fid1, '%d,%.2f,%.8f,%s,%.2f,%.2f,%.2f,%.2f\n', ...
            memberNum, lengthInch, forceOz, tOrC, pcrit, pcritUncert,
maxBuckling, maxBucklingUncert);

    end

    fclose(fid1);

    fprintf('\nCSV output written to ExtrussTrussValues.csv\n');

    fid2 = fopen('ExcriticalValues.csv', 'w');

    fprintf(fid2, 'ExcriticalValues\n');

    fprintf(fid2, 'Critical Member Number,Critical Member Length (in),Critical
Pcrit (oz),Critical Pcrit Uncertainty,Max Critical Buckling Load (oz),Max
Critical Buckling Load Uncertainty,Truss Structure Cost ($),Load-to-Cost Ratio
(oz/$)\n');

    critLength = memberLengths(criticalIdx);

    critPcrit = P_crit(criticalIdx);

    critPcritUncert = P_crit_uncert(criticalIdx);

    maxCritBuckling = critPcrit;

    maxCritBucklingUncert = critPcritUncert;

    fprintf(fid2, '%d,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.6f\n', ...
        criticalIdx, critLength, critPcrit, critPcritUncert, ...
        maxCritBuckling, maxCritBucklingUncert, trussStructureCost,
loadToCostRatio);

    fclose(fid2);

    fprintf('CSV output written to ExcriticalValues.csv\n');

    fprintf('\n=== Cost Analysis ===\n');

    fprintf('Total member length: %.2f inches\n', totalLengthInches);

    fprintf('Number of joints: %d\n', numJoints);

    fprintf('Truss structure cost: $%.2f (EK301 currency)\n',
trussStructureCost);

```

```
fprintf('Test load: %.2f oz\n', totalLoadOz);

fprintf('Critical failure load: %.2f ± %.2f oz\n', failureLoadOz,
failureLoadOz_uncert);

fprintf('Load-to-cost ratio (based on FAILURE load): %.6f ± %.6f oz/$\n',
loadToCostRatio, loadToCostRatio_uncert);

fprintf('Factor of Safety relative to test load: 2.09 ± 0.37\n', ...
failureLoadOz/totalLoadOz, failureLoadOz_uncert/totalLoadOz);

end
```